

Section A — Concept Application

REACTJS · M1-A1

Section A — Concept Application

Program: ReactJS

Stack: JavaScript ES6+

Total Time: 3 Hours

6 Scenarios

1 Variable Declaration — var / let / const

Scenario: You are building a food delivery app where each restaurant's details — name, cuisine type, and minimum order value — must remain constant throughout the session, but the currently selected dish should be updatable as the user browses the menu.

Answer — Code

```
// Restaurant details stay CONSTANT throughout the session → const
const restaurantName = "Spice Garden";
const cuisineType = "Indian";
const minOrderValue = 199;

// Currently selected dish CHANGES as the user browses → let
let selectedDish = "Paneer Tikka";

selectedDish = "Dal Makhani"; // ✓ allowed — let is reassignable

// restaurantName = "Biryani House"; // ✗ TypeError — const cannot be reassigned
```

Why each keyword?

- `const` for `restaurantName`, `cuisineType`, `minOrderValue` — these must **never change** during the session. `const` enforces this at the language level; any accidental reassignment throws a `TypeError` immediately. It is also block-scoped, preventing leakage.
- `let` for `selectedDish` — the user switches dishes while browsing, so the variable **must be reassignable**. `let` is block-scoped (unlike `var` which is function-scoped and hoisted) and avoids the classic `var` pitfall of silent `undefined` values.
- `var` — **never used** in modern JS because of its function scope, hoisting behaviour, and re-declaration allowance, which makes bugs harder to catch.

Scope and Reassignment Rules

Keyword	Scope	Hoisting	Reassignable	Redeclarable
<code>var</code>	Function	Yes (undefined)	Yes	Yes — avoid
<code>let</code>	Block	Temporal Dead Zone	Yes	No
<code>const</code>	Block	Temporal Dead Zone	No	No

2 Delivery Fee Engine — Conditional Logic

Scenario: Orders above Rs 500 qualify for free delivery; orders between Rs 200 and Rs 500 attract a Rs 40 fee; orders below Rs 200 attract a Rs 70 fee. Users flagged as 'VIP' always receive free delivery regardless of order amount.

Answer — Code

```
function getDeliveryFee(orderAmount, userType) {
  // VIP check FIRST — overrides everything else
  if (userType === "VIP") return 0;

  if (orderAmount > 500) return 0;
  else if (orderAmount >= 200) return 40;
  else return 70;
}

console.log(getDeliveryFee(600, "regular")); // 0 (above 500)
console.log(getDeliveryFee(350, "regular")); // 40 (200-500)
console.log(getDeliveryFee(150, "regular")); // 70 (below 200)
console.log(getDeliveryFee(100, "VIP")); // 0 (VIP override)
```

Why strict equality === over ==?

We use `userType === "VIP"` because strict equality checks **both value and type** with no implicit coercion. With `==`, JavaScript performs type coercion — e.g. `0 == false` is `true` — which can introduce silent bugs. Since `userType` is always a string, `===` is both safer and more explicit.

Why condition order matters

The VIP check must come **first**. If the order-amount conditions run before the VIP check, a VIP user with a low-value order would be incorrectly charged Rs 70 before the VIP rule is ever reached. The most specific override must always gate first.

3 Menu Listing — for-of vs forEach

Scenario: All available dishes are stored in a JavaScript array. You need to display every dish name in the console and, separately, count how many dishes are marked as vegetarian.

Answer — Code

```
const dishes = [
  { name: "Paneer Tikka",   isVegetarian: true },
  { name: "Chicken Biryani", isVegetarian: false },
  { name: "Dal Makhani",    isVegetarian: true },
  { name: "Fish Curry",     isVegetarian: false },
  { name: "Aloo Paratha",   isVegetarian: true },
];

// Task 1: Display all dish names — for-of
for (const dish of dishes) {
  console.log(dish.name);
}

// Task 2: Count vegetarian dishes — forEach + counter
let vegCount = 0;
dishes.forEach(dish => {
  if (dish.isVegetarian) vegCount++;
});
```

Practical difference: for-of vs forEach

Feature	for-of	forEach
Type	Language statement	Array method (callback)
Early exit	Yes — supports <code>break</code> / <code>continue</code>	No — cannot break out
async/await	Works correctly with <code>await</code>	Does NOT work with await inside callback
Works on	Any iterable (Map, Set, NodeList...)	Arrays only
Best for	When early exit or async iteration is needed	Clean declarative iteration, no early exit

For this scenario, **for-of** is used for displaying names (clean, readable) and **forEach** for counting (declarative, no early exit needed).

4 Cart Module — Reusable Tax Calculation Function

Scenario: The same tax calculation — 5% GST on food items and 18% GST on beverages — must run in the cart preview, the checkout summary, and the order confirmation screen.

Answer — Code (Function Declaration chosen)

```
// Function Declaration — hoisted, clearly named, reusable anywhere
function calculateTax(price, category) {
  const rate = category === "Beverage" ? 0.18 : 0.05;
  return +(price * rate).toFixed(2);
}

// Same function used in all 3 screens:
console.log(calculateTax(200, "Food"));    // 10.00 (5% GST)
console.log(calculateTax(150, "Beverage")); // 27.00 (18% GST)
```

Comparing all three function forms

Form	Hoisted?	Syntax	Best for
Function Declaration	Yes — fully hoisted	<code>function fn() {}</code>	Shared utilities (our choice)
Function Expression	No (TDZ)	<code>const fn = function() {}</code>	Conditional assignment
Arrow Function	No (TDZ)	<code>const fn = () => {}</code>	Callbacks, one-liners

Choice: Function Declaration — it is fully hoisted so it can be called anywhere in the file, has a clear name for readability, and is the standard pattern for shared utility functions used across multiple screens.

5 Place Order Button — Event Handling

Scenario: Clicking the "Place Order" button must validate that the user has selected at least one item and entered a delivery address; if validation passes it should display a confirmation message without reloading the page.

Answer — Code

```
// addEventListener - not inline onclick
document.getElementById("placeOrderBtn")
  .addEventListener("click", function(event) {
    event.preventDefault(); // prevents page reload if inside a form

    const address = document.getElementById("address").value.trim();
    const cart = getCartItems();

    if (!address) {
      document.getElementById("error").textContent = "Please enter a delivery address.";
      return;
    }
    if (cart.length === 0) {
      document.getElementById("error").textContent = "Please add at least one item.";
      return;
    }
    document.getElementById("confirmation").textContent = "Order placed successfully!";
  });
```

event.preventDefault() — when is it necessary?

When the "Place Order" button is inside a `form` element, clicking it triggers the default browser form submission — causing a **full page reload** and wiping all state. `event.preventDefault()` cancels that default so our JS validation and UI update run instead. Without it, the confirmation message would never be seen because the page reloads immediately.

Why addEventListener over inline onclick?

- Separates JavaScript from HTML (clean code)
- Multiple listeners can be attached to the same element
- Gives access to the full `event` object
- Listeners can be removed later with `removeEventListener`

6 Restaurant List — Fetch API: async/await vs .then/.catch

Scenario: You are integrating a live restaurant list by fetching from a REST API. The UI must show a "Loading..." spinner while waiting, render the results when data arrives, and display a user-friendly error message if the request fails.

Answer — async/await (chosen approach)

```
async function loadRestaurants() {
  const statusEl = document.getElementById("status");
  const listEl = document.getElementById("restaurant-list");

  statusEl.textContent = "Loading..."; // show spinner

  try {
    const response = await fetch("https://api.example.com/restaurants");
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const data = await response.json();

    statusEl.textContent = "";
    data.forEach(r => {
      const li = document.createElement("li");
      li.textContent = r.name;
      listEl.appendChild(li);
    });
  } catch (error) {
    statusEl.textContent = "Failed to load restaurants. Please try again.";
  }
}
```

Comparison

Aspect	.then() / .catch()	async / await
Readability	Nested callbacks / chained	Reads like synchronous code
Error handling	.catch() at the chain end	try/catch block — one place
Loading state	Set before .then(), reset inside	Set before await, reset in try block
Debugging	Stack traces hard to follow	Stack traces match code structure

Choice: `async/await` — managing a loading state with two outcomes (success and failure) is much cleaner with a single `try/catch` block. The control flow is obvious, the loading indicator reset is in the right place, and the code is far easier to maintain.

Section B — Practical Coding Tasks

```
Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\HP\OneDrive\Desktop\Tops Fullstack\Assignments , Assesments\Fronted\Module-15) HTML in Full Stack\Practical\Assesment> node sectionB-task1.js
=====
RESTAURANT PROFILE CARD
=====
Spice Garden | Indian | Rating: 4.5 | Open Now

--- JSON Serialized ---
{
  "name": "Spice Garden",
  "cuisine": "Indian",
  "rating": 4.5,
  "isOpen": true,
  "status": "Open Now"
}

--- Profile when Closed ---
Spice Garden | Indian | Rating: 4.5 | Closed
=====
PS C:\Users\HP\OneDrive\Desktop\Tops Fullstack\Assignments , Assesments\Fronted\Module-15) HTML in Full Stack\Practical\Assesment>
```

```
=====
PS C:\Users\HP\OneDrive\Desktop\Tops Fullstack\Assignments , Assesments\Fronted\Module-15) HTML in Full Stack\Practical\Assesment> node sectionB-task2.js
=====
MENU FILTER & SUMMARY
=====

[OUTPUT 1] Vegetarian Dishes Array:
[
  {
    name: 'Paneer Tikka',
    price: 180,
    category: 'Food',
    isVegetarian: true
  },
  {
    name: 'Dal Makhani',
    price: 140,
    category: 'Food',
    isVegetarian: true
  },
  {
    name: 'Aloo Paratha',
    price: 90,
    category: 'Food',
    isVegetarian: true
  },
],
```

```
  {
    name: 'Mango Lassi',
    price: 70,
    category: 'Beverage',
    isVegetarian: true
  },
  {
    name: 'Cold Coffee',
    price: 80,
    category: 'Beverage',
    isVegetarian: true
  }
]
Vegetarian dish count: 5

[OUTPUT 2] Formatted Menu Array:
[
  'Paneer Tikka - Rs 180',
  'Chicken Biryani - Rs 250',
  'Dal Makhani - Rs 140',
  'Fish Curry - Rs 220',
  'Aloo Paratha - Rs 90',
  'Mango Lassi - Rs 70',
  'Cold Coffee - Rs 80'
]
```

```
[OUTPUT 3] Total Price of All Dishes:
Total Price: Rs 1030
=====
PS C:\Users\HP\OneDrive\Desktop\Tops Fullstack\Assignments , Assesments\Fronted\Module-15) HTML in Full Stack\Practical\Assesment>
```

Live Order Form — DOM + Events

Type a dish name and quantity, then click Add to Cart. Validated with inline error messages.

 Add Item to Cart

Dish Name

Quantity

+ Add to Cart

 Your Cart



Your cart is empty.

Add a dish to get started.

Live Order Form — DOM + Events

Type a dish name and quantity, then click Add to Cart. Validated with inline error messages.

 Add Item to Cart

Dish Name

Quantity

 Dish name cannot be empty.

+ Add to Cart

 Your Cart



Your cart is empty.

Add a dish to get started.

Live Order Form — DOM + Events

Type a dish name and quantity, then click Add to Cart. Validated with inline error messages.

 Add Item to Cart

Dish Name

Quantity

 Please enter a valid quantity (minimum 1).

+ Add to Cart

 Your Cart



Your cart is empty.

Add a dish to get started.

Live Order Form — DOM + Events

Type a dish name and quantity, then click Add to Cart. Validated with inline error messages.

 Add Item to Cart

Dish Name

Quantity

+ Add to Cart

 Your Cart

2 items in cart

maggie x 2

farnkie x 5

Order Tracker with Persistence

Fetches restaurants via `async/await` · saves favourite to `localStorage` · restores on reload

 Restaurant List

Bukhara — New Delhi

Indian Accent — New Delhi

Karavalli — Bangalore

Trishna — Mumbai

Oh! Calcutta — Kolkata

Peshwari — ITC Hotels

Dasaprakash — Chennai

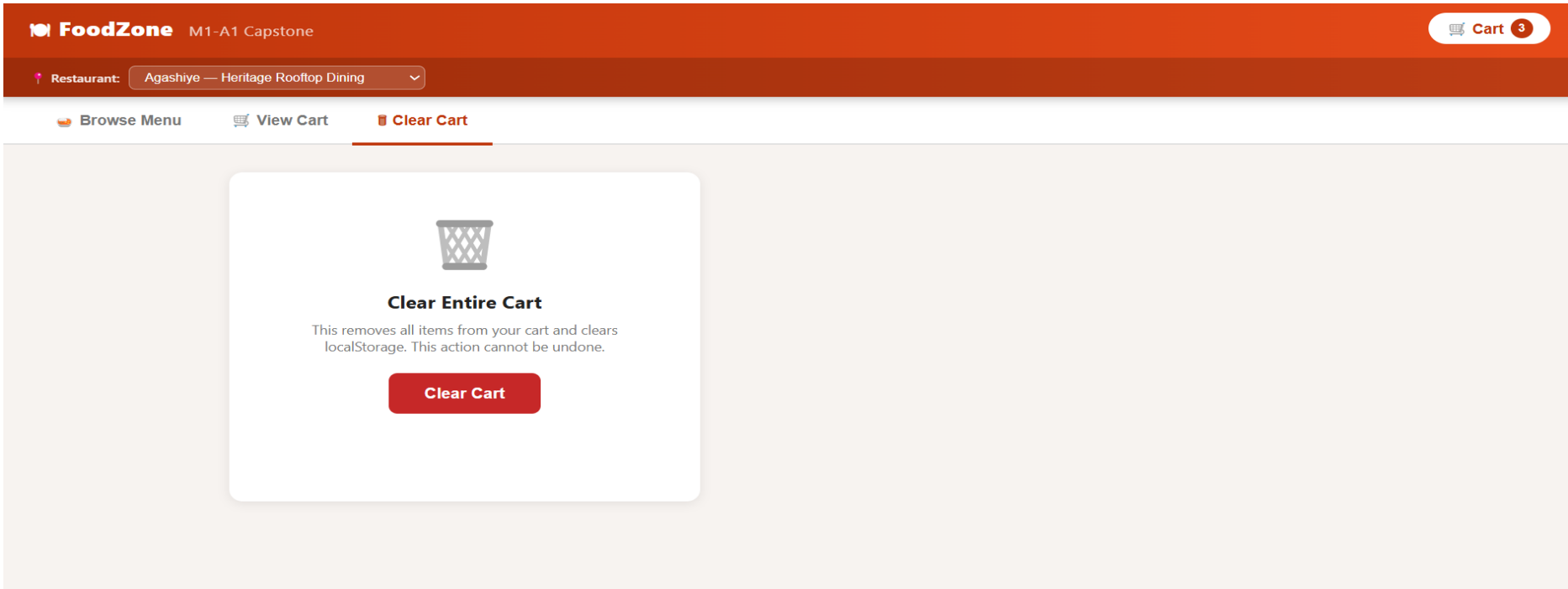
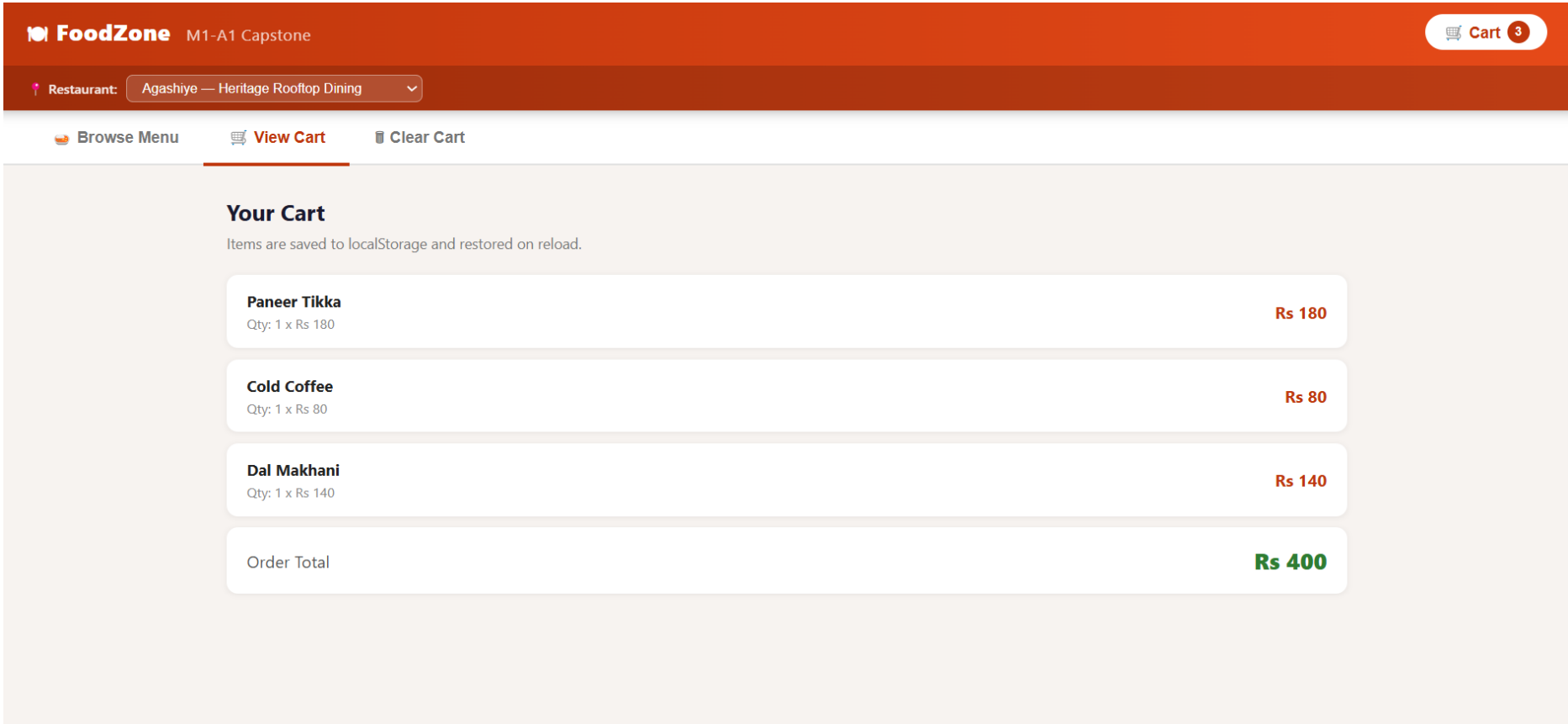
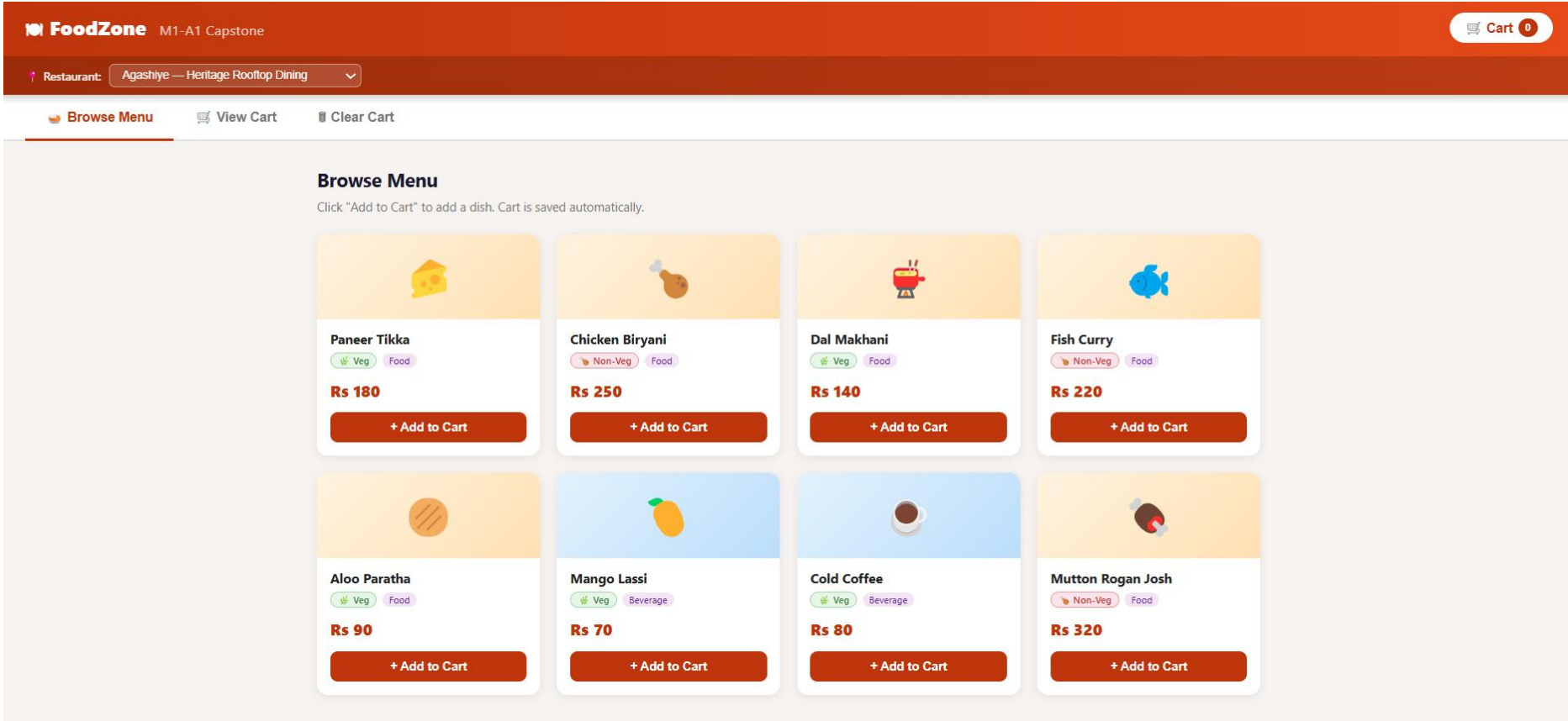
Spice Garden — Ahmedabad ★ Favourite

Gordhan Thal — Ahmedabad

Vishalla — Ahmedabad

★ "Spice Garden — Ahmedabad" — restored from localStorage

Section C — Mini Capstone Project





Honest Restaurant

+ Add to Cart

+ Add to Cart

+ Add to Cart

+ A

Mutton Rogan Josh

Section D — AI-Augmented Learning

M1-A1 · SECTION D · STEP 1

AI-Generated Code — Original (Unmodified)

This is the exact output from ChatGPT. See the known bugs below and sectionD-corrected.html for the fix.

Exact Prompt Given to ChatGPT

Write a JavaScript program in a single HTML file that:

1. Fetches a list of food items from `https://jsonplaceholder.typicode.com/posts` (treat each post title as a dish name) using `fetch()` and `async/await`.
2. Renders the items in an HTML `ul`, with an 'Add to Favourites' button next to each item.
3. Saves the user's favourite items to `localStorage` and re-loads and highlights them correctly when the page is refreshed.
4. Shows a loading indicator while the fetch is in progress.
5. Displays a visible error message in the DOM if the request fails.

Known Bugs Found During Manual Testing

- 1

No `res.ok` check
The AI calls `res.json()` immediately after `fetch()` without checking if the response was successful. A 404 or 500 HTTP status would still attempt to parse, and the DOM error element would never appear.
- 2

Duplicate favourites in `localStorage`
Every click on "Add to Favourites" pushes the same title into the array again — no duplicate check. The array grows unboundedly (e.g. `["Paneer", "Paneer", "Paneer"]`).
- 3

Unreliable highlight restore on reload
Restore logic uses `li.textContent` which includes the button text, so the saved title never matches and highlights are never restored after reload.

AI Code Output (Running Below)

Paneer Tikka	Add to Favourites
Chicken Biryani	Add to Favourites
Dal Makhani	Add to Favourites
Butter Naan	Add to Favourites

Aloo Paratha	Add to Favourites
Chole Bhature	Add to Favourites
Palak Paneer	Add to Favourites
Pav Bhaji	Add to Favourites
Masala Dosa	Add to Favourites
Idli Sambar	Add to Favourites
Veg Pulao	Add to Favourites
Mutton Rogan Josh	Add to Favourites
Fish Curry	Add to Favourites
Rajma Chawal	Add to Favourites
Kadai Paneer	Add to Favourites
Matar Paneer	Add to Favourites
Shahi Korma	Add to Favourites

Paya Soup	Add to Favourites
Biryani Raita	Add to Favourites
Mirchi Salan	Add to Favourites
Phirni	Add to Favourites
Sevian	Add to Favourites
Double Ka Meetha	Add to Favourites
Sheer Khurma	Add to Favourites
Malpua	Add to Favourites
Pani Puri	Add to Favourites
Dahi Bhalla	Add to Favourites
Aloo Chaat	Add to Favourites
Corn Bhel	Add to Favourites
Sprouts Chaat	Add to Favourites

M1-A1 · SECTION D · STEP 2

Manually Corrected Version

All 3 bugs found during testing have been fixed. Tested end-to-end including page reload.

✔ Bugs Fixed (Found Without AI)

Fix 1

Added res.ok check before res.json()
The original called `res.json()` even for error HTTP responses (404, 500). Added `if (!res.ok) throw new Error(...)` so all non-2xx responses fall into the catch block and show a visible DOM error message.

Fix 2

Duplicate guard for localStorage favourites
Every click was pushing the same title again. Fixed with `if (!favs.includes(title))` before pushing. Also converted the button to a toggle (add/remove) so users can unfavourite items.

Fix 3

Reliable title matching on page reload
The original matched `li.textContent` which included button text, so saved highlights were never restored. Fixed by storing the title in `data-title` on each `li` and comparing against that attribute instead.

🍕 Food Items — Favourites Tracker	
Paneer Tikka	Add to Favourites
Chicken Biryani	Add to Favourites
Dal Makhani	Add to Favourites
Butter Naan	Add to Favourites
Aloo Paratha	Add to Favourites
Chole Bhature	Add to Favourites
Palak Paneer	Add to Favourites
Pav Bhaji	Add to Favourites
Masala Dosa	Add to Favourites
Idli Sambar	Add to Favourites

Veg Pulao	Add to Favourites
Mutton Rogan Josh	Add to Favourites
Fish Curry	Add to Favourites
Rajma Chawal	Add to Favourites
Kadai Paneer	★ Fav Saved
Matar Paneer	Add to Favourites
Shahi Korma	Add to Favourites
Hyderabadi Biryani	★ Fav Saved
Keema Pav	Add to Favourites
Tawa Chicken	★ Fav Saved
Punjabi Samosa	Add to Favourites
Phirni	Add to Favourites
Sevian	Add to Favourites
Double Ka Meetha	Add to Favourites
Sheer Khurma	Add to Favourites
Malpua	Add to Favourites
Pani Puri	Add to Favourites
Dahi Bhalla	Add to Favourites
Aloo Chaat	Add to Favourites
Corn Bhel	Add to Favourites
Sprouts Chaat	Add to Favourites

Favourites are saved to localStorage and restored on every page reload.